

An Extension of the L^AT_EX-Theorem Environment*

Wolfgang May[†]
Institut für Informatik,
Universität Göttingen
Germany

Andreas Schedler[§]

2011/08/15

Abstract

`ntheorem.sty` is a package for handling theorem-like environments. Additionally to several features for defining the layout of theorem-like environments which can be regarded to be standard requirements for a theorem-package, it provides solutions for two related problems: placement of endmarks and generation of lists of theorem-like environments.

In contrast to former approaches, it solves the problem of setting endmarks of theorem-like environments (theorems, definitions, examples, and proofs) *automatically* at the right positions, even if the environment ends with a `displaymath` or (even nested) list environments, it also copes with the `amsmath` package. This is done in the same manner as the handling of labels by using the `.aux` file.

It also introduces the generation of lists of theorem-like environments in the same manner as `listoffigures`. Additionally, more comfortable referencing is supported.

After running L^AT_EX several times (depending on the complexity of references, in general, three runs are sufficient), the endmarks are set correctly, and theoremlists are generated.

Since `ntheorem.sty` uses the standard L^AT_EX `\newtheorem` command, existing documents can be switched to `ntheorem.sty` without having to change the `.tex` file. Also, it is compatible with L^AT_EX files using `theorem.sty` written by Frank Mittelbach.

*This file has version number 1.33, last revised 2011/08/15.

[†]`may@informatik.uni-goettingen.de`

[§]`ntheorem@andreas-schedler.net`

Contents

1	Introduction	3
2	The User-Interface	4
2.1	How to include the package	4
2.2	Defining New Theorem Sets	4
2.3	Defining the Layout of Theorem Sets	5
2.3.1	Parameters for Individual Sets	5
2.3.2	Font Selection	7
2.3.3	Predefined theorem styles	7
2.3.4	Default Setting	7
2.3.5	Deprecated: Skips until Version 1.32	8
2.3.6	A Standard Set of Theorems	8
2.3.7	Framed and Boxed Theorems	9
2.3.8	Customization and Local Settings	10
2.4	Generating Theoremlists	10
2.4.1	Defining the List Layout	11
2.4.2	Writing Extra Stuff to the Theorem File	11
2.5	For Experts: Defining Layout Styles	12
2.5.1	Defining New Theorem Layouts	12
2.5.2	Defining New Theorem List Layouts	13
2.6	Setting End Marks	13
2.7	Extended Referencing Features	14
2.8	Miscellaneous	14
3	Possible Interferences	15
3.1	Interfering Document Options.	15
3.2	Combination with amslatex.	15
3.2.1	amsmath	15
3.2.2	amsthm	16
3.3	Babel	16
3.4	Hyperref	16
3.5	Mathtools and Empheq	16
4	Examples	17
4.1	Extended Referencing Features	23
4.2	Framed and Shaded Theorems	24
4.3	Lists of Theorems and Friends	25
5	The End Mark Algorithm	28
5.1	The Idea	28
5.2	The Realization	28

6	Problems and Questions	30
6.1	Known Limitations	30
6.2	Known “Bugs” and Problems	31
6.3	Open Questions	31
7	Code Documentation	32
7.1	Documentation of the Macros	32
7.1.1	Thmmarks-Related Stuff	32
7.1.2	Option leqno to Thmmarks	40
7.1.3	Option fleqn to Thmmarks	40
7.1.4	Extended Referencing Facilities	41
7.1.5	Option amsmath to Thmmarks	44
7.1.6	Theorem-Layout Stuff	50
7.1.7	Theorem-Environment Handling Stuff	57
7.1.8	Framed and Boxed Theorems	68
7.1.9	Generation of Theorem Lists	69
7.1.10	Auxiliary macros	77
7.1.11	Other Things	78
7.2	The Standard Configuration	79
8	History and Acknowledgements	80
8.1	The endmark-Story (Wolfgang May)	80
8.2	Lists, Lists, Lists (Andreas Schedler)	80
8.3	Let’s come together	81
8.4	Acknowledgements	86

1 Introduction

For our purposes here, “theorems” are labelled enunciations, often set off from the main text by extra space and a font change. Theorems, corollaries, conjectures, definitions, examples, remarks, and proofs are all instances of “theorems”. The “header” of these structures is composed of the type of the structure (such as `THEOREM` or `REMARK`), a number which serializes the instances of the same type throughout the document, and an optional name (such as “Correctness Theorem”).

The layout of theorems can be changed by parameters as the fonts of the header and the body, the way how to arrange the headers, the indentation, and the way of numbering it. Confronted with these requirements, `theorem.sty`, a style for dealing with theorem layout was developed by Frank Mittelbach which was the standard theorem-environment for long time.

But then the desire for additional features like “endmarks” and “theorem-lists” arose. Two extensions of `theorem.sty` were developed: One for handling endmarks, `thmmarks.sty` and one for generating lists, `newthm.sty`.

Thus, Frank Mittelbach suggested to combine the new features into one “standard-to-be” package. And now, here it is.

2 The User-Interface

2.1 How to include the package

The package `ntheorem.sty` is included by

```
\usepackage[<options>]{ntheorem},
```

where the optional parameter *<options>* selects predefined configurations and special requirements.

The following *<options>* are available by now, concerning partially independent issues:

Predefined environments: (see Section 2.3.6) With `[standard]` and `[no-config]`, it can be chosen, if and what file is used for activating a (user-defined) standard set of theorem environments.

Fancy boxes around theorems: The `[framed]` option allows to use `framed.sty` that provides boxes even across pagebreaks.

Activation of endmarks: `[thmmarks]` enables the automatical placement of endmarks (see 2.3); when using the `amsmath`-package, `[thmmarks]` must be complemented by `[amsmath]` (see Section 3.2).

Activation of extended reference features: `[thref]` enables the extended reference features (see Section 4.1); when using the `amsmath`-package, `[thref]` must be complemented by `[amsmath]` (see Section 3.2).

Compatibility with `amsthm`: option `[amsthm]` provides compatibility with the theorem-layout commands of the `amsthm`-package (see Section 3.2).

Compatibility with `hyperref`: option `[hyperref]` provides compability with the `hyperref`-package (see Section 3.4).

The package itself loads `ifthen.sty`.

2.2 Defining New Theorem Sets

`\newtheorem` The syntax and semantics is exactly the same as in standard L^AT_EX: the command `\newtheorem` defines a new “theorem set” or “theorem-like structure”. Two required arguments name the new environment set and give the text to be typeset with each instance of the new “set”, while an optional argument determines how the “set” is enumerated:

`\newtheorem{foo}{bar}` The theorem set `foo` (whose name is `bar`) uses its own counter.

`\newtheorem{foo2}[foo]{bar2}` The theorem set `foo2` (printed name `bar2`) uses the same counter as the theorem set `foo`.

`\newtheorem{foo3}{bar}[section]` The theorem set `foo3` (printed name `bar`) is enumerated within the counter `section`, i.e. with every new `\section` the enumeration begins again with 1, and the enumeration is composed from the section-number and the theorem counter itself.

For every environment $\langle name \rangle$ defined by `\newtheorem`, *two* environments $\langle name \rangle$ and $\langle name* \rangle$ are defined. In the main document, they have exactly the same effect, but the latter causes no entry in the respective list of theorems (cf. `\section` and `\section*`), see also Section 2.4.

`\renewtheorem` Theorem sets can be redefined by `\renewtheorem`, with the same arguments as explained for `\newtheorem`. When redefining a theorem set, the counter is not re-initialized.

2.3 Defining the Layout of Theorem Sets

For theorem-like environments, the user can set parameters by setting several switches and then calling `\newtheorem`. The layout of a theorem set is defined with the values of the switches at the time `\newtheorem` is called.

2.3.1 Parameters for Individual Sets

The layout of individual theorem sets can be further determined by switches controlling the appearance of the headers and the header-body-layout:

<code>\theoremstyle</code>	• <code>\theoremstyle{<style>}</code>: The general structure of the theorem layout is defined via its <code>\theoremstyle</code>. <code>\nttheorem</code> provides several predefined styles including those of Frank Mittelbach's <code>theorem.sty</code> (cf. Section 2.3.3. Additional styles can be defined by <code>\newtheoremstyle</code> (cf. Section 2.5.1).
<code>\theoremheaderfont</code>	• <code>\theoremheaderfont{<fontcmds>}</code>: The theorem header is set in the font specified by <code><fontcmds></code>. In contrast to <code>theorem.sty</code>, <code>\theoremheaderfont</code> can be set individually for each environment type.
<code>\theorembodyfont</code>	• <code>\theorembodyfont{<fontcmds>}</code>: The theorem body is set in the font specified by <code><fontcmds></code>.
<code>\theoremnumbering</code>	• <code>\theoremnumbering{<style>}</code> specifies the appearance of the numbering of the theorem set. Possible $\langle styles \rangle$ are <code>arabic</code> (default), <code>alph</code>, <code>Alph</code>, <code>roman</code>, <code>Roman</code>, <code>greek</code>, <code>Greek</code>, and <code>fnsymbol</code>.

Clearly, if a theorem-environment uses the counter of another environment type, also the numbering style of that environment is used.

- `\theoremseparator`
 - `\theoremseparator{⟨thing⟩}`: `⟨thing⟩` separates the header from the body of the theorem-environment. E.g., `⟨thing⟩` can be “:” or “.”.
- `\theorempreskip`
`\theorempostskip`
 - `\theorempreskip{⟨skip⟩}` and `\theorempostskip{⟨skip⟩}` can be used to specify the vertical space before/after the theorem environment (note that Section 4.2 that allows framed and shaded theorems also defines additional skip parameters). The arguments are rubber lengths, (‘skips’), and therefore can contain **plus** and **minus** parts. (Note that these parameters changed with version 1.32; see Section 2.3.5.)
- `\theoremindent`
 - `\theoremindent⟨dimen⟩` can be used to indent the theorem wrt. the surrounding text (note that `\theoremindent` is specified without `{...}`).
 - ! It’s a ‘(dimen)’, so the user shouldn’t try to specify a **plus** or **minus** part, because this leads to an error.
- `\theoremsymbol`
 - `\theoremsymbol{⟨thing⟩}`: This is only active if `ntheorem.sty` is loaded with option `[thmmarks]`. `⟨thing⟩` is set as an endmark at the end of every instance of the environment. If no symbol should appear, say `\theoremsymbol{}`.

The above properties carry over to all subsequent `\newtheorem` statements until they are set differently. Initially, they have default values.

The following two properties apply only to the very next `\newtheorem` and are then automatically reset:

- `\theoremprework`
 - `\theoremprework{⟨thing⟩}`: `⟨thing⟩` is performed before starting the theorem structure. E.g., `⟨thing⟩` can be `\bigskip\hrule\leavevmode`. If the vertical space after your `theoremprework` does not look as intended, try to put `\leavevmode` at its end (as in the above example).
- `\theorempostwork`
 - `\theorempostwork{⟨thing⟩}`: `⟨thing⟩` is performed after finishing the theorem structure. E.g., `⟨thing⟩` can be `\hrule`.

The flexibility provided by these command should relieve the users from the ugly hacking in `\newtheorem` to fit most of the requirements stated by publishers or supervisors.

- `\theoremclass` With the command `\theoremclass{⟨theorem-type⟩}` (where `⟨theorem-type⟩` must be an already defined theorem type), these parameters can be set to the values which were used when `\newtheorem` was called for `⟨theorem-type⟩`. With `\theoremclass{LaTeX}`, the standard $\text{\LaTeX}$ layout can be chosen.

2.3.2 Font Selection

From the document structuring point of view, theorem environments are regarded as special parts inside a document. Furthermore, the theorem header is only a distinguished part of a theorem environment. Thus, `\theoremheaderfont` inherits characteristics of `\theorembodyfont` which also inherits in characteristics of the font of the surrounding environment. Thus, if for example `\theorembodyfont` is `\itshape` and `\theoremheaderfont` is `\bfseries` the font selected for the header will have the characteristics ‘bold extended italic’. If this is not desired, the corresponding property has to be explicitly overwritten in `\theoremheaderfont`, e.g. by `\theoremheaderfont{\normalfont\bfseries}`

2.3.3 Predefined theorem styles

The following theorem styles are predefined, covering those from `theorem.sty`:

<code>plain</code>	This theorem style emulates the original L ^A T _E X definition, except that additionally the parameters <code>\theorem...skipamount</code> are used.
<code>break</code>	In this style, the theorem header is followed by a line break.
<code>change</code>	Header number and text are interchanged, without a line break.
<code>changebreak</code>	Like <code>change</code> , but with a line break after the header.
<code>margin</code>	The number is set in the left margin, without a line break.
<code>marginbreak</code>	Like <code>margin</code> , but with a line break after the header.
<code>nonumberplain</code>	Like <code>plain</code> , without number (e.g. for proofs).
<code>nonumberbreak</code>	Like <code>break</code> , without number.
<code>empty</code>	No number, no name. Only the optional argument is typeset.

2.3.4 Default Setting

If no option is given, i.e. `ntheorem.sty` is loaded by `\usepackage{ntheorem.sty}`, the following default is set up:

```
\theoremstyle{plain},  
\theoremheaderfont{\normalfont\bfseries} and  
\theorembodyfont{\itshape},
```

```

\theoremseparator{},
\theorempreskip{\topsep},
\theorempostskip{\topsep}, where \topsep refers to the space
that LATEX inserts above and below lists,
\theoremindent0cm,
\theoremnumbering{arabic},
\theoremsymbol{}.

```

Thus, by only saying `\newtheorem{...}{...}`, the user gets the same layout as in standard L^AT_EX.

2.3.5 Deprecated: Skips until Version 1.32

`\theorempreskipamount` Until version 1.31, there was only a simplified handling of vertical space before/after theorems that did not consider framed and shaded theorems (that have been introduced with v1.21). `\theorempreskipamount<skip>` and `\theorempostskipamount<skip>` defined, respectively, the spacing before and after such an environment (note that both are specified without `{...}`). These parameters applied for all theorem sets and can be manipulated with the ordinary length macros. They are rubber lengths, (‘skips’), and therefore can contain **plus** and **minus** parts.

`\theorempostskipamount` Unchanged, older L^AT_EX sources that used these commands yield the same output as before since the new skip scheme described in Section 2.3.1 is only activated if one of its commands is used. Otherwise, the old scheme is applied.

2.3.6 A Standard Set of Theorems

A standard configuration of theorem sets is provided within the file `ntheorem.std`, which will be included by the option `[standard]`. It uses the `amssymb` and `latexsym` (automatically loaded) packages and defines the following sets:

Theorems: `Theorem`, `Lemma`, `Proposition`, `Corollary`, `Satz`, `Korollar`,

Definitions: `Definition`,

Examples: `Example`, `Beispiel`,

Remarks: `Anmerkung`, `Bemerkung`, `Remark`,

Proofs: `Proof` and `Beweis`.

These theorem sets seem to be the most frequently used environments in english and german documents.

The layout is defined to be `theoremstyle plain`, `bodyfont \itshape`, `Headerfont \bfseries`, and `endmark (theoremsymbol) \ensuremath{_{\Box}}` for

all theorem-like environments¹. For the definition-, remark- and example-like sets, the above setting is used, except bodyfont `\upshape`. The proof-like sets are handled a bit differently. There, the layout is defined as theoremstyle `nonumberplain`, bodyfont `\upshape`, headerfont `\scshape` and endmark `\ensuremath{\_\blacksquare}`. For a more detailed information look at `ntheorem.std` or at the code-section.

2.3.7 Framed and Boxed Theorems

With the advent of the `framed` package (by Donald Arseneau) in 2001, a feature that has often been asked for for `ntheorem` could be implemented: theorems that are framed, or that are put into a colored box. It requires to load the `framed` package; shaded theorems also require the `pstricks` package. Frames and colored boxes are orthogonal to the existing theoremstyles – thus, they can be combined in arbitrary ways.

`\newframedtheorem` A theorem type can be framed by defining it by

```
\newframedtheorem{...}{...}
```

with the same parameters as usually for `\newtheorem`. Note that the use of the `framed` package also allows to have longer theorems across a page break framed (in this case, by default, there are horizontal lines before and after the page break; this can even be circumvented by combining with `mdframed` package (since 2010)).

`\newshadedtheorem`

The same ideas hold for theorems in shaded boxes. The declaration

```
\newshadedtheorem{...}{...}
```

declares a theorem environment that is shaded. By default, the background color is `gray`. This can be changed by defining

```
\shadecolor{<color>}
```

before declaring the theorem type. Note that later declarations of other shaded theorem types can use another `shadecolor`.

By default, the box is given as a `\psframebox` (see `pstricks` package) with `shadecolor` as `linecolor` and `fillcolor`. All these parameters can be changed by setting

```
\def\theoremframecommand{<any box command>}
```

before declaring the theorem type (for examples, the user is referred to section 4).

For using `pdflatex` (where `pstricks` is not available), e.g. `\usepackage{color}` and `\theoremframecommand{\colorbox[rgb]{1,.9,.9}}` can be used.

¹Note, that `mathmode` is ensured for the symbol.

Vertical Spacing of Framed Theorems The New Skip Scheme introduced with version 1.32 allows a detailed specification of vertical space also for framed theorems (specified individually for each theorem class):

- `\theorempreskip{⟨skip⟩}` and `\theorempostskip{⟨skip⟩}` have no effect for framed theorems.

`\theoremframepreskip`
`\theoremframepostskip`

- `\theoremframepreskip{⟨skip⟩}` and `\theoremframepostskip{⟨skip⟩}` can be used to specify the vertical space before/after the frame/box.

`\theoreminframepreskip`
`\theoreminframepostskip`

- `\theoreminframepreskip{⟨skip⟩}` and `\theoreminframepostskip{⟨skip⟩}` can be used to specify the vertical space around the theorem text *inside* the frame/box.

- The arguments of the above commands are rubber lengths, (‘skips’), and therefore can contain **plus** and **minus** parts.
- the default values of all above skips is `\topsep`, i.e., the space $\text{\LaTeX}$ normally inserts before/after lists.

Old Skip Scheme (until v 1.31): `\theorempreskipamount` and `\theorempostskipamount` are applied *inside* the frame/box. To obtain vertical space *before* and *after* the frame/box in versions 1.30–v.1.31, `\theoremframepreskipamount` and `\theoremframepostskipamount` could be used (both defined by default to 0pt) analogously (i.e., they are also common to all theorem types.)

2.3.8 Customization and Local Settings

Since the user should not change `ntheorem.std`, we’ve added the possibility to use an own configuration-file. If one places the file `ntheorem.cfg` in the path searched by $\text{\TeX}$, this file is read automatically (if `[standard]` is not given). The usage of `ntheorem.cfg` can be prevented by the `[noconfig]` option. Thus, just a copy of `ntheorem.std` to `ntheorem.cfg` must be made which then can freely be modified by the user. Note, that if a configuration-file exists, this will always be used (I.e. with option `standard` and an existing configuration-file, the `.cfg` file will be used and the `.std` file won’t).

2.4 Generating Theorem lists

`\listtheorems` Similar to the $\text{\LaTeX}$ command `\listoffigures`, any theorem set defined with a `\newtheorem` statement may be listed at any place in your document by

`\listtheorems{⟨list⟩}`

The argument $\langle list \rangle$ is a comma-separated list of the theorem sets to be listed. For a theorem set $\langle name \rangle$, only the instances are listed which are instantiated by `\begin{ $\langle name \rangle$ }`. Those instantiated by `\begin{ $\langle name \rangle$ }*` are omitted (cf. `\section` and `\section*`).

For example, `\listtheorems{Corollary,Lemma}` leads to a list of all instances of one of the theorem sets “Corollary” or “Lemma”. Note, that the set name given to the command is the first argument which is specified by `\newtheorem` which is also the one to be used in `\begin{theorem} ... \end{theorem}`. If `\listtheorems` is called for a set name which is not defined via `\newtheorem`, the user is informed that a list is generated, but there will be no typeset output at all.

Note that in contrast to similar L^AT_EX commands like `\listoffigures` etc. there is no automatically created heading. Users have to write it themselves – but are free to choose what they want to have.

2.4.1 Defining the List Layout

`\theoremlisttype` Theoremlists can be formatted in different ways. Analogous to theorem layout, there are several predefined types which can be selected by

`\theoremlisttype{ $\langle type \rangle$ }`

The following four $\langle type \rangle$ s are available (for examples, the user is referred to section 4).

all List any theorem of the specified set by number, (optional) name and pagenumber. This one is also the default value.

allname Like **all**, additionally with leading theoremname.

opt Analogous to **all**, but only the theorems which have an optional name are listed.

optname Like **opt**, with leading theoremname.

2.4.2 Writing Extra Stuff to the Theorem File

Similar to `\addcontentsline` and `\addtocontents`, additional entries to theoremlists are supported. Since entries to theoremlists are a bit more intricate than entries to the lists maintained by standard L^AT_EX `\addcontentsline` and `\addtocontents` cannot be used in a straightforward way².

`\addtheoremline` Analogous to `\addcontentsline`, an extra entry for a theorem list can be made by

²for a theorem, its number has to be stored explicitly since different theorem sets can use the same counter. Also, it is optional to reset the counter for each section.

`\addtheoremline{⟨name⟩}{⟨text⟩}`

where $\langle name \rangle$ is the name of a valid theorem set and $\langle text \rangle$ is the text, which should appear in the list. For example,

`\addtheoremline{Example}{Extra Entry with number}`

generates an entry with the following characteristics:

- The Label of the theorem “Example” is used.
- The current value of the counter for “Example” is used
- The current pagenumber is used.
- The specified text is the optional text for the theorem.

Thus, the above command has the same effect as it would be for

`\begin{Example}[Extra Entry with number] \end{Example}`

except, that there would be no output of the theorem, and the counter isn’t advanced.

`\addtheoremline*` Alternatively you can use

`\addtheoremline*{Example}{Extra Entry}`

which is the same as above, except that the entry appears without number. Sometimes, e.g. for long lists, special control sequences (e.g. a pagebreak) or additional text should be inserted into a list. This is done by

`\addtotheoremfile`

`\addtotheoremfile[⟨name⟩]{⟨text⟩}`

where $\langle name \rangle$ is the name of a theorem set and $\langle text \rangle$ is the text to be written into the theorem file. If the optional argument $\langle name \rangle$ is omitted, the given text is inserted in every list, otherwise it is only inserted for the given theorem set.

2.5 For Experts: Defining Layout Styles

2.5.1 Defining New Theorem Layouts

`\newtheoremstyle` Additional layout styles for theorems can be defined by

`\newtheoremstyle{⟨name⟩}{⟨head⟩}{⟨opt-head⟩}.`

After this, `\theoremstyle{⟨name⟩}` is a valid `\theoremstyle`. Here, $\langle head \rangle$ has to be a statement using two arguments, **##1**, containing the keyword, and **##2**, containing the number. $\langle opt-head \rangle$ has to be a statement using three arguments where the additional argument **##3** contains the optional parameter.

Since L^AT_EX implements theorem-like environments by `\trivlists`, both header declarations must be of the form `\item[... \theorem@headerfont ...]...`, where the dotted parts can be formulated by the user. If there are some statements producing output after the `\item[...]`, you have to care about implicit spaces.

Because of the `@`, if `\newtheoremstyle` is used in a `.tex` file, it has to be put between `\makeatletter` and `\makeatother`.

For details, look at the code documentation or the definitions of the predefined theoremstyles.

`\renewtheoremstyle` Theorem styles can be redefined by `\renewtheoremstyle`, with the same arguments as explained for `\newtheoremstyle`.

2.5.2 Defining New Theorem List Layouts

`\newtheoremlisttype` Analogous, additional layouts for theorem lists can be defined by

`\newtheoremlisttype{<name>}{<start>}{<line> }{<end>}`.

The first argument, `<name>`, is the name of the listtype, which can be used as a valid `\theoremlisttype`. `<start>` is the sequence of commands to be executed at the very beginning of the list. Corresponding, `<end>` will be executed at the end of the list. These two are set to do nothing in the standard-types. `<line>` is the part to be called for every entry of the list. It has to be a statement using four arguments: `##1` will be replaced with the name of the theorem, `##2` with the number, `##3` with the theorem's optional text and `##4` with the page number.

WARNING: Self-defined Layouts will break with the `hyperref`-package.

`\renewtheoremlisttype` Theorem list types can be redefined by `\renewtheoremlisttype`, with the same arguments as explained for `\newtheoremlisttype`.

2.6 Setting End Marks

The automatic placement of endmarks is activated by calling `ntheorem.sty` with the option `[thmmarks]`. Since then, the endmarks are set automatically, there are only a few commands for dealing with very special situations.

`\qed` If in a single environment, the user wants to replace the standard endmark by
`\qedsymbol` some other, this can be done by saying `\qed`, if `\qedsymbol` has been defined by `\qedsymbol{<something>}` (in option standard, `\qedsymbol` is defined to be the symbol used for proofs, since a potential use of this features is to close trivial corollaries without explicitly proving them).

Additionally, if in a single environment of a theorem set, that is defined without an endmark, the user wants to set an endmark, this is done with `\qedsymbol` and `\qed` as described above. `\qedsymbol` can be redefined everywhere in the document.

`\NoEndMark` On the other hand, if in some situation, the user decides to set the endmark
`\TheoremSymbol`

manually (e.g. inside a figure or a minipage), the automatic handling can be turned off by `\NoEndMark` for the current environment. Then – assumed that the current environment is of type $\langle name \rangle$, the endmark can manually be set by just saying `\langle name \rangle Symbol`. Note that there must be no empty line in the input before the `\end{theorem}`, since then, the end mark is ignored (cf. Theorem 3 in Section 4).

2.7 Extended Referencing Features

The extended referencing features are activated by calling `ntheorem.sty` with the option `[thref]`.

Often, when writing a paper, one changes propositions into theorems, theorems into corollaries, lemmata into remarks and so on. Then, it is necessary to adjust also the references, i.e., from “`see Proposition~\ref{completeness}`” to “`see Theorem~\ref{completeness}`”. For relieving the user from this burden, the type of the respective labeled entities can be associated with the label itself:

`\label{\langle label \rangle}[\langle type \rangle]`

associates the type $\langle type \rangle$ with $\langle label \rangle$.

This task is automated for theorem-like environments:

`\begin{Theorem}[\langle name \rangle]\label{\langle label \rangle}`

is equivalent to

`\begin{Theorem}[\langle name \rangle]\label{\langle label \rangle}[Theorem]`

`\thref` The additional information is used by

`\thref{\langle label \rangle}`

which outputs the respective environment-type *and* the number, e.g., “Theorem 42”. Note that $\text{\LaTeX}$ has to be run twice after changing labels (similar to getting references OK; in the intermediate run, warnings about undefined reference types can occur).

The `[thref]` option interferes with the `babel` package, thus in this case, `ntheorem` has to be loaded *after* `babel`. It also interferes with `amsmath`; see Section 3.2.

2.8 Miscellaneous

Inside a theorem-like environment $\langle env \rangle$, the name given as optional argument is accessible by `\langle env \rangle name`.

3 Possible Interferences

Since `ntheorem` reimplements the handling of theorem-environments completely, it is incompatible with every package also concerning those macros. Additionally, the `thmmarks` algorithm for placing endmarks requires modifications of several environments (cf. Section 7). Thus, environments which are reimplemented or additionally defined by document options or styles are not covered by the endmark algorithm of `ntheorem.sty`.

The `[thref]` option changes the `\label` command and the treatment of labels when reading the `.aux` file. Thus it is potentially incompatible with all packages also changing `\label` (or `\newlabel`). Compatibility with babel's `\newlabel` is achieved if babel is loaded before `ntheorem`.

3.1 Interfering Document Options.

`ntheorem.sty` also copes with the usual document options `leqno` and `fleqn`³. If one of those options is used in the `\documentclass` declaration, it is automatically recognized by the `thmmarks` part of `ntheorem.sty`.

If one of those options is not used in `\documentclass`, but with `amsmath` (see next section), it must not be specified for `ntheorem`, since all `amsmath` environments detect this option by themselves.

3.2 Combination with `amslatex`.

`ntheorem.sty` interferes with `amsmath.sty` and `amsthm.sty`.

Note, that the LaTeX `amstex` package `amstex.sty` (L^AT_EX 2.09) is obsolete and you should use `amsmath` and `amstext` for L^AT_EX 2_ε instead. Up to `ntheorem-1.18`, it is compatible with `amsmath-1.x`. Since `ntheorem-1.19`, it is (hopefully) compatible with `amsmath-2.x`.

We would be happy if someone knowing and using `amsmath` would join the development and maintenance of this style.

3.2.1 `amsmath`

Compatibility with `amsmath` (end marks for math environments, and handling of labels in math environments) is provided in the option `[amsmath]`, (i.e., if `\usepackage{amsmath}` is used then

- `\usepackage[thmmarks]{ntheorem}` must be completed to `\usepackage[amsmath,thmmarks]{ntheorem}`), and also
- `\usepackage[thref]{ntheorem}` must be completed to `\usepackage[amsmath,thref]{ntheorem}`).

³although for `fleqn` and long formulas reaching to the right margin, equation numbers and endmarks can be smashed over the formula since `fleqn` does not use `\eqno` for controlling the setting of the equation number.

Note, that `amsmath` has to be loaded *before* `ntheorem` since the definitions have to be overwritten.

3.2.2 `amsthm`

`amsthm.sty` conflicts with the definition of theorem layouts in `theorem.sty`, some features of `amsthm.sty` have been incorporated into option `[amsthm]` which has to be used *instead of* `\usepackage{amsthm}`.

The option provides theoremstyles `plain`, `definition`, and `remark`, and a `proof` environment as in `amsthm.sty`.

The `\newtheorem*` command is defined even without this option. Note that `\newtheorem*` always switches to the nonnumbered version of the current theoremstyle which thus must be defined.

The command `\newtheoremstyle` is not taken over from `amsthm.sty`. Also, `\swapnumbers` is not implemented. Here, the user has to express his definitions by the `\newtheoremstyle` command provided by `ntheorem.sty`, including the use of `\theoremheaderfont` and `\theorembodyfont`. The options `[amsthm]` and `[standard]` are in conflict since they both define an environment `proof`.

Thus, we recommend not to use `amsthm`, since the features for defining theorem-like environments in `ntheorem.sty`—following `theorem.sty`—seem to be more intuitive and user-friendly.

3.3 Babel

The `[thref]` option interferes with the `babel` package, thus in case that `babel` is used, `ntheorem` has to be loaded *after* `babel`.

3.4 Hyperref

Since `hyperref` redefines the L^AT_EX `\contentsline`-command, it breaks with `ntheorem` below version 1.17. Since version 1.17, the option `[hyperref]` makes `ntheorem` work with `hyperref`. The entries of theoremlists then act as hyperlinks to the actual theorems. Version 1.31 incorporated some bugfixes wrt. `hyperref` for theorem lists and for the `thref` option. One should always load `\usepackage{hyperref}` *before* the first use of `\newtheorem` to obtain correct handling and referencing of counters.

WARNING: The definition and redefinition of Theorem List Layouts (see Section 2.5.2) isn't yet working with the `hyperref`-package.

3.5 Mathtools and Empheq

The `mathtools` and `empheq` packages should be loaded *before* `ntheorem` as follows:


```
\usepackage[ntheorem]{empheq} % this loads amsmath as well
\usepackage[thmmarks,amsmath]{ntheorem}
```

Note that `empheq` provides an enhanced vertical placement of the endmarks (see the paragraph on `ntheorem` in the documentation of the `empheq` package) in math environments.

4 Examples

The setting is as follows.

- For Theorems:

```
\theoremstyle{marginbreak}
\theoremheaderfont{\normalfont\bfseries}\theorembodyfont{\slshape}
\theoremsymbol{\ensuremath{\diamondsuit}}
\theoremseparator{:}
\newtheorem{Theorem}{Theorem}
```

- For Lemmas:

```
\theoremstyle{changebreak}
\theoremsymbol{\ensuremath{\heartsuit}}
\theoremindent0.5cm
\theoremnumbering{greek}
\newtheorem{Lemma}{Lemma}
```

- For Corollaries:

```
\theoremindent0cm
\theoremsymbol{\ensuremath{\spadesuit}}
\theoremnumbering{arabic}
\newtheorem{Corollary}[Theorem]{Corollary}
```

- For Examples:

```
\theoremstyle{change}
\theorembodyfont{\upshape}
\theoremsymbol{\ensuremath{\ast}}
\theoremseparator{}
\newtheorem{Example}{Example}
```

- For Definitions:

```
\theoremstyle{plain}
\theoremsymbol{\ensuremath{\clubsuit}}
\theoremseparator{.}
\theoremprework{\bigskip\hrule}
\theorempostwork{\hrule\bigskip}
\newtheorem{Definition}{Definition}
```

- For Proofs (note that `\theoremprework` and `\theoremwork` are automatically reset with the next `\newtheorem` – proofs do not have lines above and below):

```
\theoremheaderfont{\sc}\theorembodyfont{\upshape}
\theoremstyle{nonumberplain}
\theoremseparator{}
\theoremsymbol{\rule{1ex}{1ex}}
\newtheorem{Proof}{Proof}
```

Note, that parts of the setting are inherited. For instance, the fonts are not reset before defining “Lemma”, so the font setting of “Theorem” is used.

1 Example (Simple one) The first example is just a text.

In the next examples, it is shown how an endmark is put at a displaymath, a single equation and both types of eqnarrays. *

1 Theorem (Long Theorem):

The examples are put into this theorem environment.

The next example will not appear in the list of examples since it is written as

```
\begin{Example*} ... \end{Example*}
```

2 Example (Ending with a displayed formula) Look, the endmark is really at the bottom of the line:

$$f^{(n)}(z) = \frac{n!}{2\pi i} \int_{\partial D} \frac{f(\zeta)}{(\zeta - z)^{n+1}} d\zeta$$

At this point, we add an additional entry without number in the Example list:

```
\addtheoremline*{Example}{Extra Entry}
```

α Lemma (Display with array):

Lemmata are indented and numbered with greek symbols. Also for displayed arrays of this form, it looks good:

```
\[ \begin{array}{l}
a = \begin{array}{t} {1} \\
first\ line \\
second\ line \\
\end{array} \\
\end{array} \%
\mbox{try to put this text in the lowest line} \end{array} \]
```

Just try to get this with the presented array structure ... without using dirty tricks, you can position the outer array either [t], [c], or [b], and you will not get the desired effect.

$$a = \begin{array}{ll} \textit{first line} & \textit{try to put this text in the lowest line} \\ \textit{second line} & \end{array} \quad \heartsuit$$

β Lemma (Equation):

For **equations**, we decided to put the endmark after the equation number, which is vertically centered. Currently, we do not know, how to get the equation number centered and the endmark at the bottom (one has to know the internal height of the math material) ... If anyone knows, please inform us.

$$\int_{\gamma} f(z) dz := \int_a^b f(\gamma(t)) \gamma'(t) dt \quad (1) \quad \heartsuit$$

With the `break-theoremstyles`, if the environment is labeled and written as

`\begin{Lemma}[Breakstyle]\label{breakstyle}`

γ Lemma (Breakstyle):

you see, there is a leading space ...

If a percent (comment) (or an explicit `\ignorespaces`) is put directly after the label, e.g.

`\begin{Lemma}[Breakstyle]\label{breakstyle}%,`

the space disappears.

From the predefined styles, this is exactly the case for the break-styles.

That's no bug, it's $\text{\LaTeX}$ -immanent.

The example goes on with an `eqnarray`:

$$f(z) = \frac{1}{2\pi i} \int_{\partial D} \frac{f(\zeta)}{\zeta - z} d\zeta \quad (2)$$

$$= \frac{1}{2\pi} \int_0^{2\pi} f(z_0 + re^{it}) dt \quad (3) \quad \heartsuit$$

PROOF (OF NOTHING)

$$\begin{aligned} f(z) &= \frac{1}{2\pi i} \int_{\partial D} \frac{f(\zeta)}{\zeta - z} d\zeta \\ &= \frac{1}{2\pi} \int_0^{2\pi} f(z_0 + re^{it}) dt \end{aligned} \quad \blacksquare$$

That's it (the end of the Theorem).

◇

If there are some environments in the same thm-environment, the last one gets the endmark:

Definition 1 (With a list).

$$\int_{\gamma} f(z) dz := \int_a^b f(\gamma(t)) \gamma'(t) dt \quad (4)$$

- you've seen, how it works for text and
- math environments,
- and it works for lists.

♣

2 Corollary (Q.E.D.):

And here is a trivial corollary, which is ended by `\qedsymbol{\textrm{q.e.d}}` and `\qed`. q.e.d

3 Example

$$f^{(n)}(z) = \frac{n!}{2\pi i} \int_{\partial D} \frac{f(\zeta)}{(\zeta - z)^{n+1}} d\zeta$$

If there is some text after an environment, the endmark is put after the text. *

The next one is done by the following sequence. Note, that `~\hfill~` is inserted to prevent L^AT_EX from using its nested list management (a verbatim is also a trivlist), i.e. this causes L^AT_EX to start the verbatim-Part in a new line.

```
\begin{Example}
~\hfill~
\begin{verbatim}
And, it also works for verbatim
... when the \end{verbatim} is in the
same line as the text ends. \end{verbatim}
~ this space is important !!
\end{Example}
```

4 Example (Using verbatim)

```
And, it also works for verbatim
... when the \end{verbatim} is in the
same line as the text ends.
```

*

There must be no empty line in the input before the `\end{theorem}` (since then, the end mark is ignored)

```
\begin{Theorem}
some text ... but no end mark

\end{Theorem}
```

3 Theorem:

some text ... but no end mark

Now, there is a corollary which should appear with a different name in the list of corollaries:

```
\begin{Corollary*}[title in text]\label{otherlabel}
...
\end{Corollary*}\addtheoremline{Corollary}{title in list}
```

4 Corollary (title in text):

let's do something weird:

*It also works in the
center
environment.*



5 Theorem (Quote):

In quote environments, the text is normally indented from left and right by the same space. The endmark is not indented from the right margin, i.e., it is typeset to the right margin of the surrounding text.



Here is an example for turning off the endmark automatics and manual handling:

```
\begin{Theorem}[Manual End Mark]\label{somelabel}
a line of text with a manually set endmark \hfill\TheoremSymbol \\
some more text, but no automatic endmark set. \NoEndMark
\end{Theorem}
```

6 Theorem (Manual End Mark):

*a line of text with a manually set endmark
some more text, but no automatic endmark set.*



Also, one should note, that `\hfill` is inserted to set the endmark at the right margin.

5 Example (Quickie) It also works for short one's. *

If you are tired of the greek numbers and the indentation for lemmata ... you can redefine it:

```
\theoremstyle{changebreak}
\theoremheaderfont{\normalfont\bfseries}\theorembodyfont{\slshape}
\theoremsymbol{\ensuremath{\heartsuit}}
\theoremsymbol{\ensuremath{\diamondsuit}}
\theoremseparator{:}
\theoremindent0.5cm
\theoremnumbering{arabic}
\renewtheorem{Lemma}{Lemma}
```

4 Lemma:

another lemma, with arabic numbering ... note that the numbering continues. ◇

the optional argument (i.e. the ‘theorem’-name) can be accessed by $\langle env \rangle$ name.

```
\begin{Theorem}[somename]
Obviously, we are in Theorem~\Theoremname.
\end{Theorem}
```

7 Theorem (somename):

Obviously, we are in Theorem somename. ◇

This feature can e.g. be used for automatically generating executable code and a commented solution sheet:

```
\begin{exercise}[quicksort]
 $\langle the exercise text \rangle$ 
\begin{verbatimwrite}{solutions/\exercisename.c}
 $\langle C-code \rangle$ 
\end{verbatimwrite}
\verbatiminput{solutions/\exercisename.c}
\end{exercise}
```

This will write the C-code to a file `solutions/quicksort.c` and type it also on the solution sheet.

Now, we define an environment `KappaTheorem` which uses the same style parameters as Theorems and is numbered together with Corollaries (Theorems are also numbered with Corollaries). Note that we define a complex header text and a complex end mark.

```
\theoremclass{Theorem}
\theoremsymbol{\ensuremath{a \atop b}}
\newtheorem{KappaTheorem}[Corollary]{\(\kappa\)-Theorem}
```

8 κ -Theorem (1st κ -Theorem):

That's the first Kappa-Theorem.

$\begin{matrix} a \\ b \end{matrix}$

4.1 Extended Referencing Features

The standard `\label` command is extended by an optional argument which is intended to contain the “name” of the structure which is labeled, allowing more comfortable referencing; e.g., this section has been started with

```
\subsection*{Extended Referencing Features}%  
\label{sec-ExtRef}[Section]
```

As already stated, for theorem-like environments the optional argument is filled in automatically, i.e.,

```
\begin{Theorem}[Manual End Mark]\label{somelabel}
```

(cf. page 21) is equivalent to

```
\begin{Theorem}[Manual End Mark]\label{somelabel}[Theorem]
```

`\thref{<label>}` additionally outputs the contents of the optional argument which has been associated with `<label>`:

```
This is \thref{sec-ExtRef}  
A theorem end mark has been set manually in \thref{somelabel}.  
A center environment has been shown in \thref{otherlabel}.  
The first Kappa-Theorem has been given in \thref{kappatheorem1}.
```

generates

```
This is Section 4.1.  
A theorem end mark has been set manually in Theorem 6. A  
center environment has been shown in Corollary 4. The first  
Kappa-Theorem has been given in  $\kappa$ -Theorem 8.
```

Here one must be careful that the handling of the optional argument is automated only for environments defined by `\newtheorem`, i.e., *not* for sectioning, equations, or enumerations.

Calling `\thref{<label>}` for a label which has been set without an optional argument can result in different unintended results: If `<label>` is not inside a theorem-like environment, an error message is obtained, otherwise the type of the surrounding theorem-like environment is output, e.g., calling `\thref{label}` then results in “Theorem *<number>*”! Additionally, currently there is no support for multiple references such as “see Theorems 5 and 7” (this would require plural-forms for different languages and handling of `\ref`-lists, probably splitting into different sublists for different environments)⁴.

⁴If someone is interested in programming this, please contact us; it seems to be algorithmically easy, but tedious.

4.2 Framed and Shaded Theorems

Framed theorem classes are defined as follows:

```
\theoremclass{Theorem}
\theoremstyle{break}
\newframedtheorem{importantTheorem}[Theorem]{Theorem}
```

defines important theorems to use the same design as for theorems (except that the break header style is used except the margin header style), number them with the same counter, and put a frame around them:

An instance is created by

```
\begin{importantTheorem}[Important Theorem]
This is an important theorem.
\end{importantTheorem}
```

Theorem 9 (Important Theorem):

This is an important theorem.



Note that all skips have their default values (e.g. `\theoreminframepreskip` is `\topsep`). More important theorems are shaded – by default in grey:

```
\theoremclass{Theorem}
\theoremstyle{break}
\newshadedtheorem{moreImportantTheorem}[Theorem]{Theorem}
\begin{moreImportantTheorem}[More Important Theorem]
This is a more important theorem.
\end{moreImportantTheorem}
```

Theorem 10 (More Important Theorem):

This is a more important theorem.



Even more important theorems are shaded in red, with 1/2cm space inside the frame before and 1 cm space after the text, but no additional space before/after the frame:

```
\theoremclass{Theorem}
\theoremstyle{break}
\theoreminframepreskip{0.5cm}
```



```

\theoremframepostskip{1cm}
\theoremframepreskip{0cm}
\theoremframepostskip{0cm}
\shadecolor{red}
\newshadedtheorem{evenMoreImportantTheorem}[Theorem]{Theorem}
\begin{evenMoreImportantTheorem}[Even More Important Theorem]
This is an even more important theorem.
\end{evenMoreImportantTheorem}

```

Theorem 11 (Even More Important Theorem):

This is an even more important theorem.



Most important theorems get a framed, blue colored box with a shadow, no space inside the frame, and 1cm before and after the frame. Here, `\def\theoremframecommand` is used:

```

\theoremclass{Theorem}
\theoremstyle{break}
\theoremframepreskip{0pt}
\theoremframepostskip{0pt}
\theoremframepreskip{1cm}
\theoremframepostskip{1cm}
\theoremstyle{break}
\def\theoremframecommand{%
  \psshadowbox[fillstyle=solid,fillcolor=blue,linecolor=black]}
\newshadedtheorem{MostImportantTheorem}[Theorem]{Theorem}
\begin{MostImportantTheorem}[Most Important Theorem]
This is a most important theorem.
\end{MostImportantTheorem}

```

Theorem 12 (Most Important Theorem):

This is a most important theorem.



4.3 Lists of Theorems and Friends

Note, that we put the following lists into the `quote`-environment to emphasize them from the surrounding text. So the lists are indented slightly at the margin.

With

```
\addtotheoremfile{Added into all theorem lists},
```

in every list, an additional line of text would be inserted. But it isn't actually done in this documentation since we want to use different list formats. Only for the list of Examples, this one is added:

```
\addtotheoremfile[Example]{Only concerning Example lists}
```

With

```
\theoremlisttype{all}
\listtheorems{Lemma},
```

all lemmas are listed:

α	Display with array	18
β	Equation	19
γ	Breakstyle	19
4		22
5		30
6		30

From the examples, only those are listed which have an optional name:

```
\theoremlisttype{opt}
\listtheorems{Example}
```

leads to

0	Extra Entry with number	12
	Extra Entry	12
1	Simple one	18
	Extra Entry	18
4	Using <code>verbatim</code>	20
5	Quickie	22

Only concerning Example lists

One should note the line *Only concerning example lists*, which was added by the `\addtotheoremfile`-statement above.

For the next list, another layout, using the `tabular`-environment, is defined:

```
\newtheoremlisttype{tab}%
{\begin{tabular*}{\linewidth}{@{}lrl@{\extracolsep{\fill}}r@{}}%
{##1&##2&##3&##4\\}%
{\end{tabular*}}}
```

Thus, by saying

```
\theoremstyle{tab}
\listtheorems{Theorem,importantTheorem,moreImportantTheorem,
              evenMoreImportantTheorem,MostImportantTheorem,Lemma},
```

theorems (of all importance levels) and lemmata are listed:

Theorem	1	Long Theorem	18
Lemma	α	Display with array	18
Lemma	β	Equation	19
Lemma	γ	Breakstyle	19
Theorem	3		21
Theorem	5	Quote	21
Theorem	6	Manual End Mark	21
Lemma	4		22
Theorem	7	somename	22
Theorem	9	Important Theorem	24
Theorem	10	More Important Theorem	24
Theorem	11	Even More Important Theorem	25
Theorem	12	Most Important Theorem	25
Theorem	13	Correctness	29
Theorem	14	Completeness	29
Lemma	5		30
Lemma	6		30
Theorem	15		31

L^AT_EX-lists can also be used to format the theoremlist. The input

```
\newtheoremstyle{list}%
{\begin{trivlist}\item}
{\item[##2 ##1:] \ ##3\dotfill ##4}%
{\end{trivlist}}
\theoremstyle{list}
\listtheorems{Corollary}
```

leads to

2 Corollary:	Q.E.D.	20
4 Corollary:	title in list	21

In this example, after the item, `\_` is used instead of `_`, because in the latter case, `\dotfill` will produce an error if the optional argument (`##3`) is missing.

5 The End Mark Algorithm

5.1 The Idea

The handling of endmarks with `thmmarks.sty` is based on the same two-pass principle as the handling of labels: the necessary information about endmarks is contained in the `.aux` file.

With `thmmarks.sty`, $\TeX$ is always aware whether it is in some theorem-like environment. There, potential positions for endmarks can be

1. at the end of simple text lines in open text,
2. at the end of displaymaths,
3. at the end of equations or equationarrays, or
4. at the end of text lines at the end of lists (or, more general, `trivlists`, such as `verbatim` or `center`).

The problem is, that in the cases (2)–(4), the endmarks has to be placed in a box which is already shipped out, when `\end{...}` is processed. Thus, in those situations, $\TeX$ needs to know from the `.aux` file, whether it has to put an endmark.

When $\TeX$ is in a theorem-like environment and comes to one of the points mentioned in (2)–(4), and the `.aux` file says that there is an endmark, then it is put there. Anyway, it maintains a counter of the potential positions of an end mark in the current theorem-like environment. When it comes to an `\end{theorem}`, it looks if it is in situation (1) (then the endmark is simply put at the end of the current line). Otherwise, the last horizontal box is already shipped out (thus it contains a situation (2)–(4)) and the endmark must be set in it. In this case, a note is written in the `.aux` file, where the endmark actually has to be set (ie, at the latest potential point for setting an endmark inside the theorem).

5.2 The Realization

Let $\langle env \rangle$ be a theorem-like environment. Then, additional to the counter $\langle env \rangle$, $\TeX$ maintains two counters `curr $\langle env \rangle$ ctr` and `end $\langle env \rangle$ ctr`. In the i th environment of type $\langle env \rangle$, `curr $\langle env \rangle$ ctr` = i (the $\LaTeX$ counter $\langle env \rangle$ cannot be used since a) environments can use the counter of other environments, and b) often counters are reinitialized inside a document). `end $\langle env \rangle$ ctr` counts the potential situations for putting an endmark inside an environment. It is set to 1 when starting an environment. Each time, when a situation (2)–(4) is reached, the command

$$\backslash\mathrm{mark}<\backslash\mathrm{thm@romannum}\{\mathrm{curr}env\mathrm{ctr}\}>\langle env \rangle<\backslash\mathrm{thm@romannum}\{\mathrm{end}\langle env \rangle\mathrm{ctr}\}>$$

is called (where `\thm@romannum` just writes the value of a counter as its roman numeral representation, e.g., 17 as xvii).
(`<\thm@romannum{curr<env>ctr}><env><\thm@romannum{end<env>ctr}>` uniquely identifies all situations (2)–(4) in a document).
If at this position an endmark has to be set,

$$\backslash\mathrm{mark}<\backslash\mathrm{thm@romannum}\{\mathrm{curr}\langle\mathrm{env}\rangle\mathrm{ctr}\}>\langle\mathrm{env}\rangle<\backslash\mathrm{thm@romannum}\{\mathrm{end}\langle\mathrm{env}\rangle\mathrm{ctr}\}>$$

is defined in the `.aux` file to be `\end<env>\Symbol`, otherwise it is undefined and simply ignored.

When $\mathrm{T}_{\mathrm{E}}\mathrm{X}$ comes to an `\end{\langle env \rangle}`, it looks if it is in situation (1). If so, the endmark is simply put at the end of the current line. Otherwise,

$$\backslash\mathrm{def}\backslash\mathrm{mark}<\backslash\mathrm{thm@romannum}\{\mathrm{currentvctr}\}>\langle\mathrm{env}\rangle\%\\<\backslash\mathrm{thm@romannum}\{\mathrm{end}\langle\mathrm{env}\rangle\mathrm{ctr}\}>\{\langle\mathrm{env}\rangle\mathrm{Symbol}\}$$

is written to the `.aux` file for setting the endmark at the latest potential position inside the theorem in the next run.

13 Theorem (Correctness):

1. For a `.tex` file, which does not contain nested theorem-like environments of the same type, in the above situation, the following holds: When compiling, at the i th situation in the j th environment of type $\langle env \rangle$, `\mark j \langle env \rangle i` is handled.

For `.tex` files which contain nested theorem-like environments of the same type, `\mark k \langle env \rangle l` is handled, where k is the number of the latest environment of type $\langle env \rangle$ which has been called at this moment, and l is the number of situations (2)–(4) which have occurred in environments of type $\langle env \rangle$ since the k th `\begin{\langle env \rangle}`.

2. When finishing an environment, either an endmark is set directly (when in a text line) or an order to put the end symbol at the latest potential position is written to the `.aux` file. $\diamond$

14 Theorem (Completeness):

The handling of endmarks is complete wrt. `plain text`, `displaymath`, `equation`, `eqnarray`, `eqnarray*`, and all environments ended by `endtrivlist`, including `center` and `verbatim`. $\diamond$

So, where can be bugs ?

- in the plain $\mathrm{T}_{\mathrm{E}}\mathrm{X}$ handling of endmarks,
- in some special situations which have not been tested yet,
- in some special environments which have not been tested yet.
- in the `amsmath` environments. We seldom use them, so we do not know their pitfalls, and we ran only general test cases.

6 Problems and Questions

6.1 Known Limitations

- Since `ntheorem.sty` uses the `.aux` file for storing information about the positions of endmarks, $\text{\LaTeX}$ must be run twice for correctly setting the endmarks.
- Since `ntheorem.sty` uses the `.aux` file for storing information about lists in the `.thm` file, a minimum of two runs is needed. If theorems move in any of these runs up to five runs can be needed to generate correct lists.
- Since we need to expand the optional argument of theorems in various ways for the lists, we decided to copy the text verbatim into the `.thm` file. Thus, if you use things like `\thesection` etc., the list won't show the correct text. Therefore you shouldn't use any command that needs to be expanded.
- In nested environments ending at the same time, only the endmark for the inner environment is set, as the following example shows:

```
\begin{Lemma}  
  Some text.  
  \begin{Proof} The Proof \end{Proof}  
\end{Lemma}
```

yields to

5 Lemma:

Some text.

PROOF The Proof

■

You can handle this by specifying something invisible after the end of the inner theorem. Then the endmark for the outer theorem is set in the next line:

```
\begin{Lemma}  
  Some text.  
  \begin{Proof} The Proof \end{Proof}~  
\end{Lemma}
```

yields to

6 Lemma:

Some text.

PROOF The Proof

■

◇

- Document option `fleqn` is problematic: `fleqn` handles equations not by `$$` but by lists (check what happens for

```
\begin{theorem} \[ displaymath \] \end{theorem}
```

in standard L^AT_EX: The `displaymath` is *not* set in an own line). Also, for long formulas, the equation number and the endmark are smashed into the formula at the right text margin.

- Naturally, `ntheorem.sty` will not work correctly in combination with other styles which change the handling of
 1. theorem-like environments, or
 2. environments concerned with the handling of endmarks, e.g. `\[...\]`, `eqnarray`, etc.

- `ntheorem.sty` is compatible with Frank Mittelbach's `theorem.sty`, which is the most widespread style for setting theorems.

It cannot be used *with* `theorem.sty`, but it can be used instead of it.

6.2 Known “Bugs” and Problems

- Ending a theorem *directly* after the text, e.g.

```
\begin{Theorem} text\end{Theorem}
```

suppresses the endmark:

15 Theorem:

text

Therefore a space or a newline should be inserted before `\end{...}`.

- With `theoremstyle break`, if the linebreak would cause ugly linebreaking in the following text, it is suppressed.

6.3 Open Questions

- For `equations`, we decided to put the endmark after the equation number, which is vertically centered. Currently, we do not know, how to get the equation number centered and the endmark at the bottom (one has to know the internal height of the math material).

- The placement of endmarks is mainly based on a check whether $\LaTeX$ is in an ordinary text line when encountering an end-of-environment. This question is *partially* answered by `\ifhmode`: In a text line, $\LaTeX$ is always in `\hmode`. But, after an `displaymath`, $\LaTeX$ is also in `\hmode`. Thus, additionally `\lastskip` is checked: after a `displaymath`, `\lastskip=0` holds. In most situations, when text has been written into a line, `\lastskip  $\neq$  0`. But, this does not hold, if the source code is of the following form: `...text\label{bla}`: then, `\lastskip=0`. In those situations, the endmark is suppressed.
?? How can it be detected whether $\LaTeX$ has just ended a `displaymath`?
- The above problem with the label: The break style enforces a linebreak by `\hfill\penalty-8000` after the `\trivlist`-item. Thus, $\TeX$ gets back into the horizontal mode. The label places a “whatsit” somewhere ... and, it seems that the “whatsit” makes $\TeX$ think that there is a line of text.

If someone has a solution to one of those questions, please inform us. (You can be sure to be mentioned in the Acknowledgements.)

7 Code Documentation

7.1 Documentation of the Macros

```
1 \typeout{Style ‘\basename’, Version \fileversion\space <\filedate>}
2 \ProvidesPackage{ntheorem}[\filedate \space\fileversion]
3 \RequirePackage{ifthen}%
4 \newif\if@thmmarks\@thmmarksfalse
5 \newif\if@thref\@threffalse
6 \newif\ifthm@inframe\thm@inframefalse
7 \newif\ifthm@tempif
```

general setup.

7.1.1 Thmmarks-Related Stuff

```
1 \DeclareOption{thmmarks}{%*****
2 \PackageInfo{\basename}{Option ‘thmmarks’ loaded}%
3 %
4 \@thmmarkstrue
5 \newcounter{endNonectr}
6 \newcounter{currNonectr}
7 \newif\ifsetendmark\setendmarktrue
```

activate placement of endmarks and define counters for upper level.

`\ifsetendmark`: true if an endmark has to be set in a complex situation which must be handled by the `.aux` file. For further comments see `\@endtheorem`.